

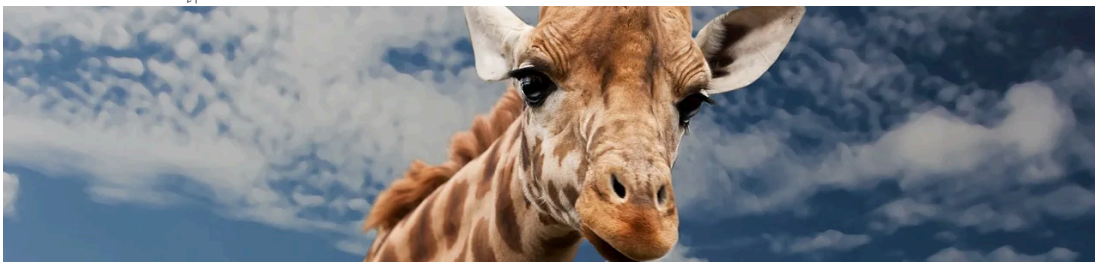
当大模型接管编程：NASA 疯狂的“反人类”编程要求，为何仍被奉为行业圣典？

作者：华卫

2025-03-19 北京

本文字数：10694 字

阅读完需：
约 35 分钟



整理 | 华卫

在软件工程领域，有些“老派”的方法和理念，是经过时间检验的真理，值得我们重新审视和学习。

大多数大型软件开发项目都会使用编码规范，旨在规定编写软件的基本规则：代码应如何构建，以及应该使用和避免哪些语言特性，尤其是在代码的正确性会对设备产生决定性影响的领域，如潜水艇、飞机、将宇航员送上同步轨道的航天器，以及距离居民区仅几公里之外的核电站等设施运行的控制代码等。

在众多编码规范中，NASA 的编码规则以其严苛性和有效性反复被提起。近期，油管博主 ThePrime Time 发布的解读 NASA 安全编码规则的视频，甚至短时间内引发了超百万观看。

特别是在 AI 编程和“氛围编程”流行的当下，重新审视严谨、可验证的编程规范，是对软件工程本质的回归。

有声音说，“老派的 NASA 编码方式是最好的方式。”也有人评价，“在 C 语言中使用这些标准的编码人员是真正的战士。”



@nullmeasure6155 3周前

The old school NASA way is the best way. In Apollo we trust.



71



回复



@krunkle5136 3周前

The coders that worked with these standards in C are true warriors.



1515



回复

NASA 程序员在编写航天设备运行代码时都遵守一套严格的规则，这套编码规则由 NASA 喷气推进实验室（JPL）首席科学家 Gerard J. Holzmann 所提出，名为《The Power of Ten – Rules for Developing Safety Critical Code1》（十

推荐阅读

Angular 16 正式发布，抢先

大前端，编程语言，Web，

基于 Vue 和 Canvas，轻
端可视化编辑器设计解析

大前端，编程语言，Web，

谷歌最好的程序员 Jeff D
种编程语言

语言 & 开发，文化 & 方法

低代码编程及其市场机遇
术内幕

文化 & 方法，语言 & 开发

从大前端“穿越”到终端，
什么技能？ | 解读终端的

编程语言，DevOps & 平台

《架构师》（2022 年 4 月

语言 & 开发，架构，编程

相关课程

极客搜索·AI版

专属技术人的一站式 AI 搜索工具

限时免费

用 DeepSeek 武装自己的
立即参与 >>

电子书



腾讯云云原

精选集 202

《2024腾讯

实践精选集》

术领域，13

例，痛点到解

立即下载

大厂实战PPT下载

基于知 大模型应用实

曹阳/中关村金 资深AI产品总监



美国航天局对于工具使用方式的规则（提到了 Python），可能是出于工程师们的期望。其他一些规则则是为了防止同一组织早期编码工作中出现的非常特定且不太可能发生的错误类型。毫不奇怪，现有编码规范对开发人员实际编写代码的行为影响甚微。许多规范最致命的方面是它们很少允许进行全面的基于工具的合规性检查。基于工具的检查很重要，因为对于大型应用程序编写的数十万行代码，手动审查通常是不可行的。”

ThePrime Time 对此表达了强烈地赞同，称“确实有很多个人偏好被写入了代码规范中。我认同目前提到的所有内容，代码就应该可靠。自动化和工具的使用应该杜绝个人偏好。”

NASA 的编码规则主要针对 C 语言，力求优化更全面检查用 C 语言编写的关键应用程序可靠性的能力。原因是，“在包括 JPL 在内的许多组织中，关键代码都是用 C 语言编写的。由于其悠久的历史，这种语言有广泛的工具支持，包括强大的源代码分析器、逻辑模型提取器、度量工具、调试器、测试支持工具，以及成熟稳定的编译器选择。因此，C 语言也是大多数已开发的编码规范的目标。”

ThePrime Time 表示，“我知道现在有很多软件开发人员，一听到用 C 语言编写安全关键代码，可能就会想‘怎么又是这个’。你们可没有像‘旅行者号’（NASA 研制的太空探测器）那样的项目，你们还不是顶尖开发者。”

此外，Holzmann 认为，“为了有效，规则集必须很小，并且必须足够清晰，以便于理解和记忆。规则必须足够具体，以便可以机械地进行检查。当然，这么小的规则集不可能涵盖所有情况，但它可以为我们提供一个立足点，对软件的可靠性和可验证性产生可衡量的影响。”因此，他将 NASA 的编码规则限制在十条。

这十条规则正在 NASA 喷气推进实验室用于关键任务软件的编写实验，取得了令人鼓舞的成果。据 Holzmann 介绍，一开始，NASA 的开发人员对遵守如此严格的限制存在合理的抵触情绪，但克服之后，他们常常发现，遵守这些规则确实有助于提高代码的清晰度、可分析性和安全性。这些规则减轻了开发人员和测试人员通过其他方式确定代码关键属性（例如终止性、有界性、内存和栈的安全使用等）的负担。“这些规则就像汽车上的安全带，起初可能有点让人不舒服，但过一段时间后，使用它们会成为习惯，不使用反而难以想象。”

ThePrime Time 最后对 NASA 编码规则给出的整体评价是，“我喜欢这份文档，即便我并非完全认同其中所有的规则。我只是很惊讶，政府机构编写的内容竟如此条理清晰。这是一份极其连贯的文档，似乎出自一位追求务实的人之手。”

不少与 NASA 工程师共事过的开发者们，都对这则 NASA 十大编码规则的解读视频深有感触：“他们的编码指南并不‘疯狂’，反而实际上相当理智。我们没有以这种方式编程才是疯狂的”，并分享了许多个人的相关经历。



@unknown6656 3周前

Misleading video title: the coding guidelines are not "insane". They're actually quite sane. We are insane for not programming that way....

159 回复

“在学习 C 语言的时候，我的教授曾为卫星编写 C 程序 / 代码。他把自己的方法教给了我们，这种方法要求我们在电脑上编程之前，先把所有内容都写在纸上。这种方式迫使我们准确理解自己正在编写的内容、内存分配等知识，还能编写出更高效的代码，并掌握相关知识。我很庆幸自己是通过这种方式学习的，因为在面试时，我能轻松地在白板上编写代码。”一名工程师说。

另一位与前 NASA 工程师共同开发过游戏的程序员透露，“他的代码是我见过的最整洁、最易读的。当时我还是一名初级程序员，仅仅通过和他一起编写代码，我就学到了很多。我们使用的是 C++ 语言，但他的编程风格更像是带有类的 C 语言。他的代码本身就很易于理解（具有自解释性），不过他仍然对自己的代码进行了注释（既有代码中的注释，也有实际的文档说明）。”

裁员潮下，工作的逻辑化？

雪梅 | 雪梅 前阿里技术总规划手册

立即下载

《中国生成式 AI 开发手册》报告发布

姜昕蔚 | InfoQ 研究总监兼

立即下载



同时有前 NASA 工程师出来现身说法道，“曾参与构建云基础设施，他们的指导原则可不是闹着玩的，代码审查简直是人间炼狱。‘严苛’这个词用来形容再贴切不过了。不过，相比我之前在电信、金融科技领域的工作经历，以及后来在其他科技公司的工作，我在 NASA 工作期间对可靠性方面的了解要多得多。”

“NASA 的编码要求太疯狂了”

对于这十条规则，Holzmann 已经声明，“为了支持强大的审查，这些规则有些严格，甚至可以说严苛。但这种权衡是有道理的。在关键时候，尤其是开发安全关键代码时，多费些功夫，遵守更严格的限制是值得的。这样我们就能更有力度地证明关键软件能按预期运行。”并且，每条规则之后都附了其被纳入的简短理由。

ThePrime Time 对这些编码规则及理由一一进行了评价和分析，以下是经不改变原意的翻译和编辑后整理出来的解读内容。

规则一：

将所有代码限制在非常简单的控制流结构中，不要使用 goto 语句、setjmp 或 longjmp 结构以及直接或间接递归。

理由：更简单的控制流意味着更强的验证能力，并且通常能提高代码的清晰度。禁止递归可能是这里最让人意外的一点。不过，如果没有递归，我们就能保证有一个无环的函数调用图，这能被代码分析器利用，还能直接帮助证明所有本应有限的执行实际上都是有限的。（注意，这条规则并不要求所有函数都有一个单一的返回点——尽管这通常也会简化控制流。不过，有很多情况下，提前返回错误是更简单的解决方案。）

ThePrime Time：我不知道间接递归是什么意思。间接递归是指两个函数相互调用吗？在实际情况中，这种情况确实会发生，而且发生过很多次。比如有一个函数需要调用另一个函数去做某些事情并进行一些检查，然后再通过某种不受你控制的方式返回结果。特别是在那些没有异步 / 等待（async/await）机制的语言里，我猜你只能阻塞线程了，对吧？规则是说如果没有异步机制，就只能阻塞线程，然后在一个while循环里处理，是这样吗？我得想想我自己使用间接递归的场景。实际上，我在处理套接字重连时就用到间接递归。具体来说，当我打开一个套接字（就像这样，打开这个套接字），重连机制会调用一个私有函数来重置状态，然后调用reconnect函数，reconnect函数会调用connect函数，当连接断开时，connect函数又会再次调用自己。从技术上讲，这就是一种间接递归。我在想，也许我可以把它改成用while循环加上等待机制，这样会不会更简单呢？现在真的让我开始思考这个问题了，这可能只是一种替代方案。

哎呀，我已经违反规则一了。不过间接递归确实是一种非常强大的无限问题解决机制，但我可以尝试不用它，对吧？我完全不介意尝试新的做法。理由很简单，“更简单的控制流意味着更强的验证能力，并且通常能提高代码的清晰度。”我认同这一点，确实看到代码里有类似这样的逻辑时会觉得有点绕。比如在一个 close 函数中调用 reconnect，然后在 reconnect 中检查是否已经启动，如果没有完成，就返回。这些语句的顺序会导致问题，所以我可以理解为什么会出现这种情况。我甚至可以说服自己接受这一点。

说实话，这是一个非常务实的观点，解释了为什么不应该使用递归。而且说句公道话，我大学三四年级的时候（不对，其实是大二），老师让我们用非递归的方式实现 AVL 树。如果你不熟悉 AVL 树，这是一种使用旋转的自平衡二叉搜索树，有四种不同的旋转：右旋、左旋、先右后左旋和先左后右旋。做起来其实很有趣，假设我们有一个二叉树，像这样：a（根节点），b 是左子节点，c 是右子节点。我们想重新组织成 b 作为根节点，a 作为左子节点，c 作为右子节点。如果你有一棵二叉树，看起来像“a b c”，你就把它重组为“a c b”，然后进行旋转。很简单直接，对吧？老师说我们要实现这个程序，但不能用递归。这对我来说是一次很棒的学习经历。

所有循环必须有固定的上限。检查工具必须能够轻易地静态证明循环的预设迭代上限不会被突破。如果无法静态证明循环的上限，就视为违反规则。

理由：没有递归且存在循环上限可以防止代码失控。当然，这条规则不适用于那些本就不打算终止的迭代（例如在进程调度器中）。在这些特殊情况下，适用相反的规则：必须能静态证明迭代不会终止。支持这条规则的一种方法是，给所有迭代次数可变的循环添加明确的上限（比如遍历链表的代码）。当超过上限时，会触发断言失败，包含失败迭代的函数将返回错误。（关于断言的使用，请参见规则 5）

ThePrime Time： 这是不是意味着不能使用像数组的 `forEach` 这样的方法呢？因为从技术上讲，其上限是根据数组动态变化的，没有固定值。还是说不能使用 `while (true)` 这种循环呢？这是一条有趣的规则。

规则三：

初始化后不要使用动态内存分配。

理由：这条规则在安全关键软件中很常见，并且出现在大多数编码规范里。原因很简单：像 `malloc` 这样的内存分配器和垃圾回收器，其行为往往不可预测，可能会对性能产生重大影响。一类显著的编码错误也源于对内存分配和释放例程的不当处理，比如忘记释放内存、释放后继续使用内存、试图分配超过实际可用的内存，以及越界访问已分配的内存等等。强制所有应用程序在固定的、预先分配好的内存区域内运行，可以避免很多这类问题，也更容易验证内存的使用情况。需要注意的是，在不使用堆内存分配的情况下，动态申请内存的唯一方法是使用栈内存。在没有递归的情况下（规则 1），可以静态推导出栈内存使用的上限，从而可以证明应用程序将始终在其预先分配的内存范围内运行。

ThePrime Time： 这么一来，JavaScript 和 Go 语言可就有点麻烦了。不过说实在的，其实在 JavaScript 和 Go 里也能做到这一点。显然，在大多数情况下是可以的。但我敢肯定，只要调用类似 `G Funk`（这里可能是随意提及的某个函数）这样的函数，它背地里肯定会分配一些你不知道的内存。当然，不是所有解释型语言都这样，这么说不太准确，不是所有解释型语言都有这个问题。

我猜这条规则意味着不能使用闭包，对吧？因为闭包会涉及到内存分配。准确地说，你得使用内存池（Arena）进行内存分配。也就是说，不能随意使用列表或字符串吗？也不是，其实可以使用列表和字符串，只是意味着所有内存分配都必须在程序开始时完成。我猜这里说的是堆内存，而不是栈内存。另外，我是这么理解的，比如说你从服务器获取一系列响应数据，你得事先分配一块足够大的内存区域，用来存储所有可能的响应数据，然后像使用环形缓冲区一样循环利用这块内存，这样就不会有额外的内存分配操作了，所有可能用到的数据都已经预先分配好了。所以一开始你就应该拥有所需的所有内存，这意味着可以使用字符串，只是得预先定义好字符串占用内存的大小。天呐，这得好好琢磨琢磨，确实很费脑筋，不过环形缓冲区的概念真的很有意思。

“在不使用堆内存分配的情况下，动态申请内存的唯一方式是使用栈内存。根据规则一，在没有递归的情况下，可以静态推导出栈内存使用的上限，这样就能证明应用程序始终在其预先分配的内存范围内运行。”这听起来有点疯狂，但其实挺酷的，仔细想想还挺有道理。

我听说在游戏开发里有这样一种做法，如果我说错了也请大家指正。在游戏开发中，每个子团队会有各自的资源预算，包括内存和 CPU 时间。在你负责的程序部分，你只能使用分配给你的那部分资源。一旦超出预算，就会有类似这样的提示：“嘿，物理模拟团队，你们用的时间太多了，能不能想想办法？”我觉得这听起来挺不错的。我知道有这么回事，我举的这个例子是希望普通的游戏开发者也能理解。就好比今年两家小的游戏工作室因为资源超支没拿到奖金，大致就是这么个情况。宽泛来讲，实际情况比在 Twitch 聊天里说的要复杂一些，但差不多就是这样。我只是以一个普通游戏开发者的角度来解释这个规则，我开发过一些小游戏，但我也知道自己还算不上专业的游戏开发者。

任何函数的长度都不应超过以标准参考格式打印在一张纸上的长度，即每行写一条语句、每行写一个声明。通常情况下，这意味着每个函数的代码行数不应超过 60 行。

理由：每个函数都应该是代码中的一个逻辑单元，可以作为一个单元来理解和验证。跨越计算机显示器多个屏幕或打印时多页的逻辑单元要难得多。过长的函数往往是代码结构不佳的表现。

ThePrime Time：好的，这挺合理的。60 行代码的空间足够你把事情弄清楚了。鲍勃大叔（Uncle Bob，著名编程大师 Robert C. Martin）规定每个函数一般只能有三到五行代码，相比之下 60 行代码算很多了。不过这里说的是打印在一张纸上，对吧？就是说代码打印在单张纸上，大概就是这个意思。注意，这其实也不算特别严格的硬性规定，但从能打印在纸上这个角度来说，它又算是个硬性规定。我觉得 60 行代码能表达很多内容，肯定有办法打破这个规则，但我感觉自己通常能轻松写出最多 60 行代码的函数，我觉得这一点都不难。没错，Ghost 标准库有数千行代码，但我不会把 Ghost 标准库当作史上最整洁、最出色的代码之一。老实说，我个人觉得 Ghost 标准库读起来真的很糟糕。

这条规则的理由是，每个函数都应该是代码中的一个逻辑单元，能够作为一个整体被理解和验证。如果一个逻辑单元跨越计算机显示器的多个屏幕，或者打印出来有好多页，理解起来就困难得多。过长的函数往往意味着代码结构不佳。我基本上同意这个观点，我觉得实际上很少能见到超过 60 行代码的函数。而且一般来说，当你遇到这样的函数时，要么是因为功能本身非常复杂，由于行为的关联性必须写在一起；要么这个函数写得很糟糕。除非你写 React 代码，我觉得我说的这些还是适用的。

规则五：

代码的断言密度平均每个函数至少应有两个断言。断言用于检查在实际执行中不应发生的异常情况。断言必须始终无副作用，并且应定义为布尔测试。当断言失败时，必须采取明确的恢复措施，例如，向执行失败断言的函数的调用者返回错误条件。任何静态检查工具能够证明永远不会失败或永远不会成立的断言都违反此规则。（也就是说，不能通过添加无用的“assert (true)”语句来满足该规则。）

理由：工业编码工作的统计数据表明，单元测试通常每编写 10 到 100 行代码就能发现至少一个缺陷。断言密度越高，拦截缺陷的几率就越大。断言的使用通常也被推荐作为强防御性编码策略的一部分。断言可用于验证函数的前置和后置条件、参数值、函数返回值以及循环不变式。由于断言无副作用，因此在测试后，可以在对性能关键的代码中有选择地禁用它们。

ThePrime Time：我很喜欢这条规则。我觉得它有很多优点，而且我觉得自己需要更多地实践这条规则。我还得继续坚持，因为就像我之前说的，我的代码库里已经有不少断言了。我确实经常使用断言，但目前我代码里的断言一旦触发，程序就会直接崩溃。比如说，当程序内部生成的消息与我预期的不一致时，我就会认为自己犯了严重错误，觉得整个程序都得“炸掉”，结果整个程序就会受到影响。有一点很棒，如果你能想出某种模糊测试策略，就能测试你的程序。你可以往程序里输入一堆随机数据，而程序里应该有防御性的语句，以确保不会触发更多的断言。这样一来，你甚至可以减少很多针对模糊测试的特定单元测试，这是不是很神奇？

下面是一个典型的断言使用示例：如果条件 C 成立，且断言 p 大于等于为真，就返回错误。假设断言定义如下：定义一个名为 `c_assert` 的断言，用于调试，当断言失败时，输出文件和行号等信息，这里设置为 `false`。在这个定义中，`file` 和 `line` 由宏预处理器预先定义，用于输出失败断言所在的文件名和行号。语法 `#e` 会将断言条件 `e` 转换为字符串，作为错误消息的一部分打印出来。在嵌入式程序代码中，通常没有地方打印错误消息，在这种情况下，对测试调试的调用会变成空操作，断言就变成了纯粹的布尔测试。这有助于从异常行为中恢复错误。

规则六：

数据对象必须在尽可能小的作用域级别声明。

理由：这条规则支持数据隐藏的基本原则。显然，如果一个对象不在作用域内，其值就不能被引用或破坏。同样，如果必须诊断一个对象的错误值，可能分配该值的语句越少，诊断问题就越容易。该规则不鼓励将变量重复用于多个不兼容的目的，这可能会使故障诊断复杂化。

ThePrime Time：有人说这只是因为 C 语言没有恰当的错误处理和语法特性，我强烈反对这种说法。实际上，断言非常有用。比如说，Tiger Beetle 项目是用 Zig 语言编写的，Zig 有恰当的错误处理工具，它有结果对象，而且其自身的错误处理结果对象能提供比标准错误更好的堆栈跟踪信息，这真的很酷。我记得在采访时，他说当时 Tiger Beetle 项目里有 8000 个断言。

没错，断言不是错误处理机制，实际上断言不是用于处理错误的，它是一种不变式，可以说是硬性终止条件。我们来看看，这和在 Zig 语言里直接返回错误有什么不同呢？在 Zig 里，你可以返回一个错误或者一个值，但这就是一种错误处理方式。Zig 里不会硬性终止程序，它必须有恢复机制。我觉得这样也挺好，无论是硬性终止，还是软性终止并搭配某种错误恢复机制，都需要构建相应的错误恢复机制。

我其实并没有完全理解规则六，除了感觉好像是在使用变量的地方定义它，这样作用域就是最小的，是这个意思吗？听起来好像是这样，这里是在说封装的概念吗？

规则七：

非 void 函数的返回值必须由每个调用函数检查，并且每个函数内部必须检查参数的有效性。

理由：这可能是最常被违反的规则，因此作为一般规则有些可疑。从严格意义上讲，这条规则意味着即使是 printf 语句和文件关闭语句的返回值也必须被检查。不过也有观点认为，如果对错误的响应与对成功的响应没有区别，那么显式检查返回值就没什么意义。这通常是调用 printf 和 close 的情况。在这种情况下，可以接受将函数返回值显式转换为 (void)——这表明程序员是有意忽略返回值，而非不小心遗漏。在更可疑的情况下，应该有注释解释为什么返回值无关紧要。不过，在大多数情况下，函数的返回值不应被忽略，尤其是在必须将错误返回值沿函数调用链向上传播的情况下。标准库因违反此规则而臭名昭著，并可能导致严重后果。例如，如果不小心执行 strlen(0)，或者使用标准 C 字符串库执行 strcat(s1, s2, -1)——结果就很糟糕。遵循这条通用规则，我们可以确保例外情况必须有合理的解释，并且机械检查器会标记违规行为。通常，遵守这条规则比解释为什么不符合规则更容易。

ThePrime Time：这实际上是我非常喜欢 Zig 的一个原因。我想 Rust 语言也有类似的情况，只不过当你忽略返回的结果或异步操作的返回值时，Rust 只是给出警告。我喜欢这条规则，我觉得这是一条很棒的规则。这是一条普遍适用的好规则。要知道，编程的很大一部分就是学习这些技巧，避免自己给自己挖坑。

规则八：

预处理器的使用必须仅限于包含头文件和简单的宏定义。不允许使用令牌粘贴、可变参数列表（省略号）和递归宏调用。所有宏必须展开为完整的语法单元。条件编译指令的使用通常也值得怀疑，但并非总是可以避免。这意味着，即使在大型软件开发项目中，除了避免同一头文件多次包含的标准样板代码外，也很少有理由使用超过一两个条件编译指令。每次此类使用都应通过基于工具的检查器标记，并在代码中说明理由。

及 C 语言中“条件编译”仅用于开发阶段的特性。C 语言在开发阶段引入宏的机制。对条件编译时提供必要的性能优化是

要。请注意，仅使用十个条件编译指令，就可能有多达 2 的 10 次方种可能的代码版本，每种版本都必须进行测试——导致所需的测试工作量大幅增加。

ThePrime Time：我认为对预处理宏保持谨慎肯定没错。我理解代码时遇到的困难，没有比处理预处理宏更多的了。预处理宏真的是最难懂的部分之一。这条规则实际上会让我们的开发工作变得更加复杂，我一点都不喜欢。有意思的是，总体来说我不喜欢预处理器。我能理解预处理器肯定会引发一堆问题，人们通常把它们叫做宏。一般来说，宏可能很有用，但它们通常也非常难以理解，理解起来特别费劲。谢天谢地 C 语言没有宏（这里表述有误，C 语言有宏，作者可能想表达宏的复杂性让人头疼）。宏是一种强大的工具，但就像所有强大的工具一样，它们非常危险。预处理器是一个强大的混淆工具，会破坏代码的清晰度，让许多基于文本的检查器感到困惑。即使手头有正式的语言定义，不受限制的预处理代码中的结构效果也极难解读。在 C 预处理器的新实现中，开发人员常常不得不借助早期的实现来解读 C 标准中的复杂定义语言。

对条件编译保持谨慎的理由同样重要。要知道，仅仅 10 个条件编译指令就可能产生多达 2 的 10 次方种代码版本，每个版本都必须进行测试，这会大幅增加所需的测试工作量。我是说，这一点非常关键。一般来说，条件编译就是一场噩梦，尽管有时又不得不使用它。我觉得这条规则务实的地方在于，它认识到虽然无法避免使用条件编译，但条件编译确实非常困难且麻烦。

没错，我猜 Rust 语言的 cargo 特性主要是因为 Rust 编译速度慢才存在的。我认识的大多数人对 Rust 二进制文件不太感兴趣，更多的是担心编译时间太慢。我敢肯定，最终 Rust 二进制文件非常重要，但就我所知，很多时候问题就出在编译速度上。正是因为编译慢，才引出了一系列问题，比如我总是会遇到这样的情况，使用 clap 时忘记添加 feature derive，然后又得去添加；使用 request 时又忘记添加 request feature 之类的，情况越来越糟。你们看过 AutoSAR 的 C 代码吗？我敢肯定那代码很糟糕。

规则九：

指针的使用应受到限制。具体来说，允许的解引用级别不超过一级。指针解引用操作不得隐藏在宏定义或 typedef 声明中。不允许使用函数指针。

理由：即使是经验丰富的程序员也容易误用指针。它们可能使程序中的数据流难以跟踪或分析，尤其是对于基于工具的静态分析器。同样，函数指针可能会严重限制静态分析器可以执行的检查类型，只有在有充分理由使用它们的情况下才应使用，并且理想情况下应提供替代方法来帮助基于工具的检查器确定控制流和函数调用层次结构。例如，如果使用函数指针，工具可能无法证明没有递归，因此必须提供替代保证来弥补分析能力的损失。

ThePrime Time：比如说，怎么处理异步相关的操作和中断呢？我觉得他们可能不处理异步操作，但我又确定他们肯定会处理。处理异步操作肯定得使用某种互斥锁，对吧？比如使用信号量互斥锁，然后还得涉及对内存的引用。异步操作具有不确定性，所以不太处理好。其实也不是完全不确定，只是你得在一定程度上进行处理。想象一下，你有一个探测器，上面有个小摄像头正在拍照。在某个时刻你拍了照，照片进行处理后存储到内存中，处理完成后会有提示。然后某些代码需要被唤醒，这不也在一定程度上涉及到函数指针吗？我猜得用互斥锁，对吧？所以我猜你会有一段代码，比如说处理拍照的代码。你得在这里进行一些操作，比如获取信号量，在 C 语言里信号量的值为 1，具体是用 lock unlock（锁定解锁）还是 lock acquire（获取锁）我记不清了。代码就停在这里等待，当照片数据传入后，代码开始处理，处理完之后再回到等待状态。

我理解这个，不过这条规则感觉更难落实。



规则十：

品级分类: 高级开发工程师。

理由：如今市场上有几种非常有效的静态源代码分析器，还有相当多的免费工具。任何软件开发工作都没有理由不使用这种现成的技术。即使对于非关键代码的开发，也应将其视为常规做法。零警告规则甚至适用于编译器或静态分析器给出错误警告的情况：如果编译器或静态分析器感到困惑，应重写导致困惑的代码，使其更简单有效。很多开发者一开始认为某个警告肯定是无效的，结果后来才意识到，由于一些不那么明显的原因，该警告实际上是合理的。早期的静态分析器，比如 lint，大多会给出无效的提示信息，这让静态分析器的名声有些不好，但现在情况已经不同了。当今最好的静态分析器速度快，并且会生成有针对性且准确的提示消息。在任何一个严肃的软件项目中，它们的使用都不应有商量余地。

ThePrime Time：说实话，我觉得这挺合理的。尤其是对于新手而言，如果你刚开始接触软件开发，就应该能够做到这一点。公平地说，我不算专业的软件开发人员，所以我能理解这一点。我觉得这真的很棒，规则 10 简直太实用了。

不过，要把这条规则应用到很多项目中可能会很困难。比如说在 JavaScript 开发中，大家都知道 JavaScript 的 lint 工具体验很差，大多数 ESLint 规则纯粹是些主观的好坏评判标准。比如在处理 Promise 时，使用 `re` 和 `reject` 作为参数实际上是不好的做法，对吧？

参考链接：

<https://www.youtube.com/watch?v=JWKadu0ks20>

声明：本文为 InfoQ 整理，不代表平台观点，未经许可禁止转载。

大会推荐：

在 AI 大模型技术如汹涌浪潮席卷软件开发领域的当下，变革与机遇交织，挑战与突破共生。2025 年 4 月 10 - 12 日，QCon 全球软件开发大会将在北京召开，以“智能融合，引领未来”为年度主题，汇聚各领域的技术先行者以及创新实践者，为行业发展拨云见日。现在报名可以享受 8 折优惠，单张门票立省 1360 元，详情可联系票务经理 18514549229 咨询。

InfoQ 极客传媒

洞悉 AI 新纪元下的机遇

直击行业痛点，解锁可复制的经验与模式

大模型正在重新定义软件

2025 北京

面向 AI 的基础设施

云上业务架构演进

大模型赋能 AIOps

多模态大模型及应用

Lakehouse

越挫越勇的大前端

海外 AI 应用创新实践

应用出海

全链路可观测实践

二类“大”模型

大模型产品创新与探索

QCon 北京站

2025.04.10-12 / 北京

扫码查看大会详情 >>

咨询购票

查看详情

评论

快抢沙发！虚位以待

发布

暂无评论

联系我们

内容投稿：editors@geekbang.com
业务合作：hezuo@geekbang.com
反馈投诉：feedback@geekbang.com
加入我们：zhaopin@geekbang.com
联系电话：010-64738142
地址：北京市朝阳区望京北路9号2幢7层A701

InfoQ 近期会议

北京 · QCon 全球软件开发大会 2025.4.10-12
上海 · AICon 全球人工智能开发与应用大会 2025.5.23-24
北京 · AICon 全球人工智能开发与应用大会 2025.6.27-28



促进软件开发及相关领域知识与创新的传播

关于我们
我要投稿
合作伙伴
加入我们
关注我们

